

REGIONAL IDENTIFICATION PREDICTION FROM FACIAL STRUCTURE

*A Major Project report submitted in partial fulfillment of the requirements for the
award of the degree of*

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

PASUNOORI KIRANMAI (22S41A0598)

PADAKANTI SAI SANGEETHA (22S41A0594)

MOHAMMED SHAKEEBUDDIN (22S41A0583)

NVV SATYANARAYANA (22S41A0591)

**Under the Guidance of
Mrs. R. APOORVA
Assistant Professor**



**Department of Computer Science & Engineering
VAAGESWARI COLLEGE OF ENGINEERING**

**Accredited by NAAC with A+ Grade
(Affiliated to JNTU Hyderabad & Approved by AICTE, New Delhi)
Ramakrishna colony, Karimnagar-505527.**

April-2026

Department of Computer Science & Engineering
VAAGESWARI COLLEGE OF ENGINEERING
Accredited by NAAC with A+ Grade
(Affiliated to JNTU Hyderabad & Approved by AICTE, New Delhi)
Ramakrishna colony, Karimnagar-505527.



CERTIFICATE

This is to certify that the project report entitled “**REGIONAL IDENTIFICATION PREDICTION FROM FACIAL STRUCTURE**” submitted by the following students in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in CSE and is a Bonafide record of the work performed by.

PASUNOORI KIRANMAI	(22S41A0598)
PADAKANTI SAI SANGEETHA	(22S41A0594)
MOHAMMED SHAKEEBUDDIN	(22S41A0583)
N.V.V. SATYANARAYANA	(22S41A0591)

Internal Guide
Mrs. R. APOORVA
Assistant Professor

Head of the department
Dr. N. CHANDRAMUOLI
Professor

Principal

Dr. CH. SRINIVAS

External Examiner

DECLARATION

We hereby declare that the project titled “**REGIONAL IDENTIFICATION PREDICTION FROM FACIAL STRUCTURE**” submitted to Vaageswari College of Engineering, affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH) for the award of the Degree of Bachelor of technology in CSE is a result of original research carried-out in this work. It is further declared that the report or any part thereof has not been previously submitted to any University or Institute for the award of degree or diploma.

1. Name of the Student: PASUNOORI KIRANMAI

Hall Ticket Number:22S41A0598

2. Name of the Student: PADAKANTI SAI SANGEETHA

Hall Ticket Number:22S41A0594

3. Name of the Student: MOHAMMED SHAKEEBUDDIN

Hall Ticket Number:22S41A0583

4. Name of the student: N.V.V. SATYANARAYANA

Hall Ticket Number:22S41A0591

Date:

ACKNOWLEDGEMENT

We wish to pay our sincere thanks to **Dr. CH. SRINIVAS**, Principal, Vaageswari College of Engineering, Karimnagar, for providing all required facilities and his support during the Project Work.

We would like to thank **Dr. N. CHANDRAMOULI**, Professor and HOD of the Computer Science and Engineering Department for his valuable suggestions during the Project Work.

We sincerely extend our thanks to the major project guide **Mrs. R. APOORVA** Assistant Professor of Computer Science and Engineering Department, for sparing his valuable in guiding the major project work and giving feedback with many useful suggestions during the major project work.

We are also conveying our heartfelt thanks to the Institute authority, Department, Library, and Laboratory staff of Vaageswari College of Engineering for their co-operation during our Project Work. We thank our beloved friends for their help and encouragement regarding the concepts and presentation.

PASUNOORI KIRANMAI	(22S41A0598)
PADAKANTI SAI SANGEETHA	(22S41A0594)
MOHAMMED SHAKEEBUDDIN	(22S41A0583)
N.V.V SATYANARAYANA	(22S41A0591)

ABSTRACT

Deep learning has become one of the most powerful technologies in computer vision, particularly for identifying complex human characteristics from facial images. Regional identification based on facial structures is an emerging area that leverages deep models to analyze patterns and variations across different populations. By training object detection models on diverse datasets, it is possible to extract key visual attributes that help differentiate regional origins. Such systems must balance high performance with fairness, ethical considerations, and careful dataset curation.

This research focuses on building a software-based regional identification system using YOLOv8 architecture. The system is designed to process facial datasets collected from seven different countries: Bhutan, Brazil, Dubai, India, Korea, Nigeria, and South Africa. The model is trained on this dataset and then applied to identify the regional category of unseen facial inputs. Emphasis is placed on accuracy, real-time performance, reproducibility, and a modular design that supports dataset management, model generation, and recognition workflows.

INDEX

	PAGE NO
CHAPTER – 1 : INTRODUCTION	1-2
CHAPTER – 2 : LITERATURE SURVEY	3 – 5
CHAPTER – 3 : REQUIREMENTS ANALYSIS	6 – 9
3.1 Existing System	6
3.2 Proposed System	6
3.3 System Requirements	7
3.4 Functional Requirements	7
3.5 Non-Functional Requirements	8
3.6 System Study	9
CHAPTER – 4 : SYSTEM DESIGN	10 – 23
4.1 Software Environment	10 – 12
4.2 System Architecture	12
4.3 Algorithm Used	11 – 13
4.4 Dataflow	13 – 14
4.3 UML Diagrams	15 – 22
CHAPTER – 5 : IMPLEMENTATION	23 – 31
5.1 Module Description	26
5.2 Methodology	26 – 27
5.3 Sample Code	27 – 31
5.4 System Testing	31
5.4.1 Types of Testing	32 – 34
CHAPTER – 6 : RESULT AND DISCUSSION	35 – 41
6.1 Screenshots	35 – 41
6.2 Result	41
CHAPTER – 7 : CONCLUSION AND FUTURE SCOPE	42 – 43
7.1 Conclusion	42
7.2 Future Scope	42 – 43
REFERENCES	44

LIST OF FIGURE

S NO	TITLE	PAGE NO
1	Django Framework	10
2	Django Backend	10
3	System Architecture	12
	Data Flow Diagram Level 0	13
4	Data Flow Diagram Level 1	14
5	Data Flow Diagram Level 2	14
6	Admin Use Case Diagram	16
7	User Use Case Diagram	16
8	Class Diagram	17
9	Sequence Diagram	18
10	Activity Diagram	19
11	System Deployment Diagram	20
12	Statechart Diagram	20
13	System Component Diagram	21
14	Object Diagram	21
15	Collaboration Diagram	22
16	SCREENSHOTS	35–41

CHAPTER-1

INTRODUCTION

1.1 AIM

To develop a deep-learning-based system capable of predicting the regional origin of facial images using YOLOv8 models trained on multi-country datasets.

1.2 OBJECTIVE

The main objective is to implement a software framework that enables training, evaluation, and deployment of a facial structure-based regional identification model, while ensuring efficient recognition, modular operation, and academic value.

1.3 PROBLEM STATEMENT

Human facial recognition systems are widely used for identity verification, but regional identification from faces remains underexplored. Traditional approaches lack the robustness and scalability to differentiate faces from diverse global populations. Existing methods often fail to generalize across varied ethnic groups due to dataset imbalance, bias, and limitations in earlier model architectures. The challenge is to design a system that efficiently trains on multi-regional data, accurately predicts unseen cases, and provides an easy-to-use software workflow for academic purposes.

1.4 OVERVIEW OF THE SYSTEM

Facial recognition is an active research field that has evolved from handcrafted feature methods to modern deep-learning pipelines. The latest architectures such as YOLOv8 provide high-speed detection capabilities and can be extended for specialized classification tasks. Their modularity and efficiency make them suitable for tasks such as regional identification, where rapid image processing and robust feature extraction are critical.

Regional prediction from facial features is challenging due to intra-class variations such as age, gender, expression, and external conditions like lighting and pose. Unlike standard identity recognition, regional classification focuses on structural and cultural facial variations across populations, requiring carefully curated and balanced

datasets. The adoption of datasets spanning multiple countries ensures exposure to diverse facial characteristics, enabling models to learn robust discriminative features.

Despite the potential of deep models, the field faces ethical and technical hurdles. Dataset biases can lead to skewed performance across groups, raising fairness concerns. Therefore, dataset design and reporting of per-region performance metrics must be a central part of system evaluation. Documenting these issues is essential for any academic project dealing with demographic attributes.

YOLOv8's strength lies in its real-time detection ability and the flexibility to incorporate classification heads. The detection stage isolates facial regions, ensuring that the classifier focuses on the most relevant parts of the image. This pipeline reduces noise from the background and improves model accuracy, making it suitable for regional identification tasks.

Another important aspect is reproducibility. The system must log dataset versions, training configurations, and model metrics. Academic projects require reproducible experiments, allowing others to replicate results under similar conditions. Implementing training pipelines with controlled seeds, standardized logging, and versioned datasets ensures scientific credibility.

Finally, while the project demonstrates technical feasibility, it emphasizes that regional prediction systems should be used only for controlled academic research. In real-world applications, caution must be exercised to prevent misuse. This ensures the project contributes knowledge to computer vision while acknowledging its limitations.

CHAPTER-2

LITERATURAL SURVEY

TITLE: YOLOv8 — Models & Usage (Documentation)

AUTHORS: Ultralytics Team

ABSTRACT: The YOLOv8 documentation by Ultralytics serves as the official technical resource for understanding and applying the YOLOv8 architecture. It describes in detail the different model variants (nano, small, medium, large, x-large), their speed-versus-accuracy tradeoffs, and provides end-to-end guidance on dataset preparation, training, validation, and inference. The documentation highlights improvements such as anchor-free detection, updated backbone networks, and modular design that allow researchers to easily adapt the architecture for tasks beyond object detection, including segmentation and classification. For academic projects like DeepEthno, these resources are crucial because they enable rapid prototyping and deployment with minimal configuration while ensuring reproducibility. The documentation also emphasizes code examples and API usage, reducing the technical barrier for researchers unfamiliar with prior YOLO versions. This makes YOLOv8 particularly suitable for academic projects requiring both efficiency and ease of integration.

PUBLICATION YEAR: 2023

TITLE: Real-Time Flying Object Detection with YOLOv8

AUTHORS: D. Reis

ABSTRACT: This paper focuses on real-time detection of flying objects, such as drones and aircraft, using YOLOv8. It emphasizes the importance of lightweight architectures for time-sensitive applications where both speed and accuracy are critical. The study compares YOLOv8 with earlier YOLO variants and other real-time object detectors, demonstrating YOLOv8's superior ability to process high-velocity targets in dynamic environments. For DeepEthno, the key insight lies in its handling of real-time processing and the emphasis on dataset augmentation to improve generalization in complex conditions. Similar principles can be applied to facial recognition and regional

identification, where the model must account for variations in pose, lighting, and occlusion. The methodology also reinforces the significance of preprocessing pipelines that ensure robust feature extraction, making YOLOv8 not just an object detector but a versatile backbone for domain-specific classification.

PUBLICATION YEAR: 2023

TITLE: Running Ultralytics Object Detection and Segmentation Models in a Few Lines of Code

AUTHORS: N. Ladi

ABSTRACT: This article presents an implementation-focused approach to running YOLOv8 and other Ultralytics models using minimal Python code. By showcasing how to load pre-trained models, apply them to datasets, and perform inference in just a few lines, it reduces the technical barrier for adoption. The work demonstrates the simplicity of transferring models into custom datasets, making it highly relevant for researchers who need to build quick prototypes. For DeepEthno, this usability is critical: admins and researchers can quickly upload new datasets, retrain YOLOv8 models on country-specific faces, and deploy updated classifiers with minimal overhead. Moreover, the paper emphasizes reproducibility by promoting best practices in using model checkpoints, saving logs, and exporting results to standardized formats. This reinforces the project's academic credibility, ensuring that models can be replicated and validated across different environments.

PUBLICATION YEAR: 2024

TITLE: Validating Privacy-Preserving Face Recognition under a Minimum Assumption

AUTHORS: Z. Zhang et al.

ABSTRACT: The paper addresses the increasing demand for privacy-preserving approaches in face recognition systems, where sensitive biometric data is often processed and stored. It introduces the concept of validating privacy-preserving systems under minimal assumptions, emphasizing the balance between preserving user confidentiality and maintaining recognition accuracy. For the DeepEthno system, which deals with regional and demographic attributes, this paper highlights important considerations for

securing data and ensuring ethical compliance. The proposed frameworks, such as Map2V, allow models to be validated in a way that respects privacy constraints while still performing complex recognition tasks. This becomes critical when dealing with multi-country datasets, where cultural and legal restrictions on biometric data handling vary significantly. The paper also points out the risks of demographic profiling and underscores the importance of ensuring that such systems are not misused beyond academic contexts.

PUBLICATION YEAR: 2024

TITLE: Surveying Racial Bias in Facial Recognition: Balancing Accuracy and Fairness

AUTHORS: A. Sumsion

ABSTRACT: This comprehensive survey examines the issue of racial bias in facial recognition systems, focusing on how imbalanced datasets lead to uneven performance across demographic groups. The author discusses fairness metrics such as disparate impact, equal error rate, and per-class accuracy, and evaluates various bias mitigation strategies including dataset balancing, augmentation, and algorithmic adjustments. For DeepEthno, which attempts to classify individuals into regional categories, this survey underscores the risks of skewed accuracy if certain groups (e.g., underrepresented regions) dominate the training set. The study also explores real-world consequences of biased recognition, such as wrongful classifications and cultural insensitivity, which are particularly relevant in the context of academic research where ethical considerations must be documented. By adopting fairness evaluation techniques recommended in this survey, DeepEthno can position itself as a responsible prototype rather than a biased recognition system.

PUBLICATION YEAR: 2024

CHAPTER-3

REQUIREMENTS ANALYSIS

3.1 EXISTING SYSTEM

Current facial recognition systems predominantly focus on identity verification and authentication, not regional classification. Some frameworks can detect demographic attributes such as age or gender, but they do not address fine-grained regional differences. Older systems rely on shallow machine learning or handcrafted features, which lack robustness under varied conditions. Newer systems may use CNN backbones but rarely offer modular tools for dataset upload, model generation, and structured recognition workflows.

DISADVANTAGES:

1. Bias and unfair performance due to unbalanced or limited datasets
2. Lack of modularity in existing tools for academic use
3. Reduced performance under noisy or uncontrolled input conditions

3.2 PROPOSED SYSTEM

The proposed system uses YOLOv8 as the backbone for face detection and classification. It integrates a structured pipeline that allows datasets from seven countries to be uploaded, validated, and used for training a classification-capable YOLOv8 model. Once trained, the model can recognize new facial images and predict their regional class. The system emphasizes modular separation of dataset management, model training, and recognition functions. It also incorporates reproducibility, ensuring that training logs, model versions, and evaluation metrics are consistently recorded for academic validation.

ADVANTAGES

1. Fast and accurate recognition using YOLOv8
2. Dataset-driven modular design with support for multiple countries
3. Academic reproducibility with logs and model versioning
4. Clear separation of dataset management and recognition processes

3.3 SYSTEM REQUIREMENTS:

3.3.1 HARDWARE REQUIREMENTS:

- **Processor** : Intel iConre
- **SSD** : 512 GB.
- **Ram** : 8 GB

3.3.2 SOFTWARE REQUIREMENTS:

- **Operating system** : Windows 11.
- **Coding Language** : python
- **Tool** : pycharm
- **Database** : mysql
- **framework** : Django
- **Technologies** : pdbc, Deep Learning

3.4 FUNCTIONAL REQUIREMENTS

Admin Functions

1. **Login** – The admin must authenticate securely before accessing the system.
2. **Upload Dataset** – The admin can upload facial datasets belonging to the 7 countries {Bhutan, Brazil, Dubai, India, Korea, Nigeria, South_Africa}.
3. **Generate Model** – The admin can initiate training of a YOLOv8 model using the uploaded datasets.
4. **Store Model** – The trained model should be saved

User Functions

1. **Registration** – Users can register with unique credentials.
2. **Login** – Users must log in to access recognition services.
3. **Upload Image** – Users can upload trained person images for recognition.
4. **Face Recognition** – The system loads the latest YOLOv8 model, runs detection & classification, and predicts the regional category.
5. **View Results** – The recognition result (predicted country with confidence score) is displayed to the user.
6. **Logout** – Users can log out after recognition.

3.5 NON-FUNCTIONAL REQUIREMENTS

1. Performance

- Training jobs must complete within reasonable timeframes depending on dataset size.
- Inference (recognition) should respond in < 3 seconds for a single image.

2. Scalability

- The system should support uploading larger datasets and retraining without requiring major architectural changes.
- Capable of handling concurrent user recognition requests.

3. Security

- Encrypt user credentials and sensitive data.
- Restrict admin-only functions (dataset upload, model training).

4. Reliability & Availability

- Ensure recognition service is available with minimal downtime.
- Provide backup mechanisms for datasets and models.

5. Usability

- Simple and intuitive interface for both admin (dataset/model) and user (image upload/recognition).
- Provide meaningful error messages (e.g., invalid file format, failed recognition).

6. Maintainability

- Modular design separating admin functions, user functions, and core recognition logic.
- Easy to update or replace the YOLOv8 model with newer versions.

7. Reproducibility

- Store model metadata (training dataset, hyperparameters, accuracy) to reproduce experiments.

8. Privacy & Ethics

- Ensure uploaded images are stored securely and deleted on user request.
- Explicitly restrict system use to **academic and research purposes only**.

3.6 SYSTEM STUDY

➤ FEASIBILITY STUDY

The feasibility of the project is analyzed in this phase and business proposal is put forth with a very general plan for the project and some cost estimates. During system analysis the feasibility study of the proposed system is to be carried out. This is to ensure that the proposed system is not a burden to the company. For feasibility analysis, some understanding of the major requirements for the system is essential.

FEASIBILITY ANALYSIS

◆ ECONOMICAL FEASIBILITY

◆ TECHNICAL FEASIBILITY

➤ ECONOMICAL FEASIBILITY

This study is carried out to check the economic impact that the system will have on the organization. The amount of fund that the company can pour into the research and development of the system is limited. The expenditures must be justified. Thus the developed system as well within the budget and this was achieved because most of the technologies used are freely available. Only the customized products had to be purchased.

➤ TECHNICAL FEASIBILITY

This study is carried out to check the technical feasibility, that is, the technical requirements of the system. Any system developed must not have a high demand on the available technical resources. This will lead to high demands on the available technical resources. This will lead to high demands being placed on the client. The developed system must have a modest requirement, as only minimal or null changes are required for implementing this system.

CHAPTER-4

SYSTEM DESIGN

4.1 SOFTWARE ENVIRONMENT

4.1.1 PYTHON

Python is a **high-level, interpreted, interactive** and **object-oriented scripting language**. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

- **Python is Interpreted:** Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive:** You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented:** Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language:** Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

4.1.2 DJANGO

Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design. Built by experienced developers, it takes care of much of the hassle of Web development, so you can focus on writing your app without needing to reinvent the wheel. It's free and open source.

Django's primary goal is to ease the creation of complex, database-driven websites. Django emphasizes reusability and "pluggability" of components, rapid

development, and the principle of don't repeat yourself. Python is used throughout, even for settings files and data models.

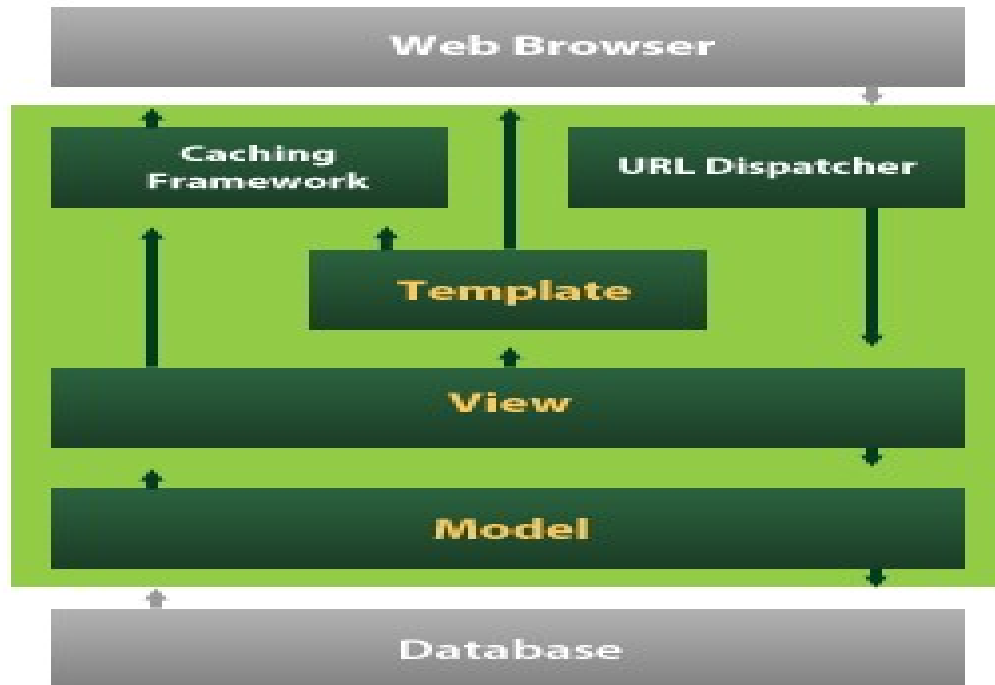


FIG 4.1.1 : DJANGO FRAMWORK

Django also provides an optional administrative [create, read, update and delete](#) interface that is generated dynamically through [introspection](#) and configured via admin models

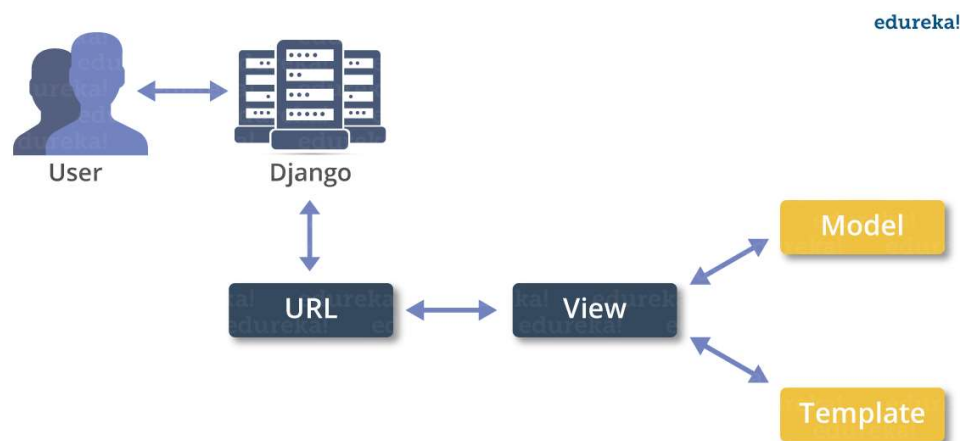


FIG 4.1.2 : DJANJO BACKEND

4.1.3 MACHINE LEARNING

- What is Machine Learning
1. Machine Learning is the science (and art) of programming computers so they can learn from data.
 2. Machine Learning is a step into the direction of artificial intelligence (AI).
 3. Machine Learning is a program that analyses data and learns to predict the outcome.
 4. Machine learning is a growing technology which enables computers to learn automatically from past data.
 5. Machine learning uses various algorithms for building mathematical models and making predictions using historical data or information. Currently, it is being used for various tasks such as image recognition, speech recognition, email filtering etc..

4.2 SYSTEM ARCHITECTURE

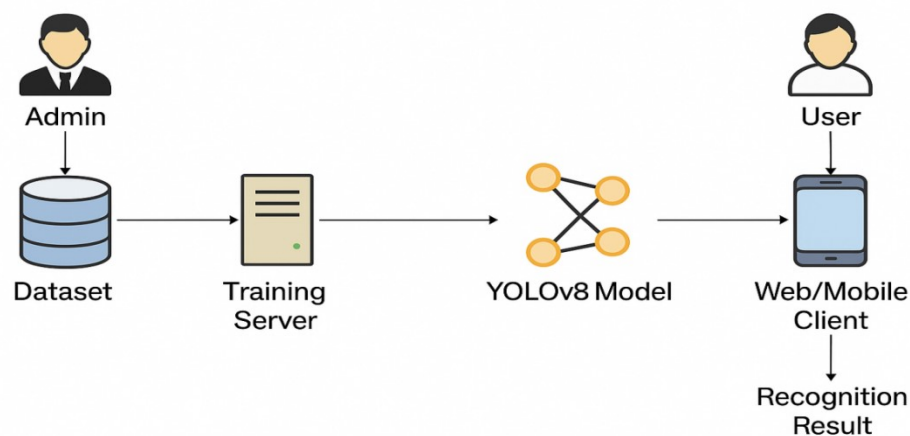


FIG 4.2: ARCHITECTURE

4.3 ALGORITHM USED

In this project, **YOLOv8** is used to **detect and classify human faces** from different countries.

We trained YOLOv8 with a custom dataset containing facial images from **seven countries** — *Bhutan, Brazil, Dubai, India, Korean, Nigeria, and South Africa*.

- During **model building**, YOLOv8 is either **loaded** (if pre-trained weights exist) or **trained** using these labeled images defined in data.yaml.
- Once trained, the model learns to recognize **unique facial patterns and features** for each country.
- When a **user uploads an image**, YOLOv8 performs **inference**, detects the face region, and predicts which country it belongs to.
- The detection results (bounding boxes and country label) are **visualized and saved** as an output image for display in the web application.

4.4 DATAFLOW

Data Flow Diagram (DFD) is a graphical representation of data flow in any system. It is capable of illustrating incoming data flow, outgoing data flow and store data. The DFD depicts both incoming and outgoing data flows and provides a high-level overview of system functionality. It is a relatively simple technique to learn and use, making it accessible for both technical and non-technical stakeholders.

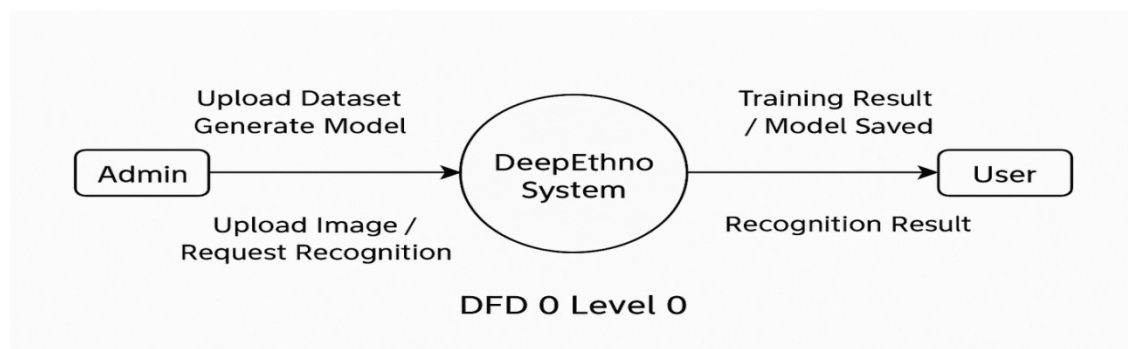


FIG 4.4.1 : DATA FLOW DIAGRAM LEVEL 0

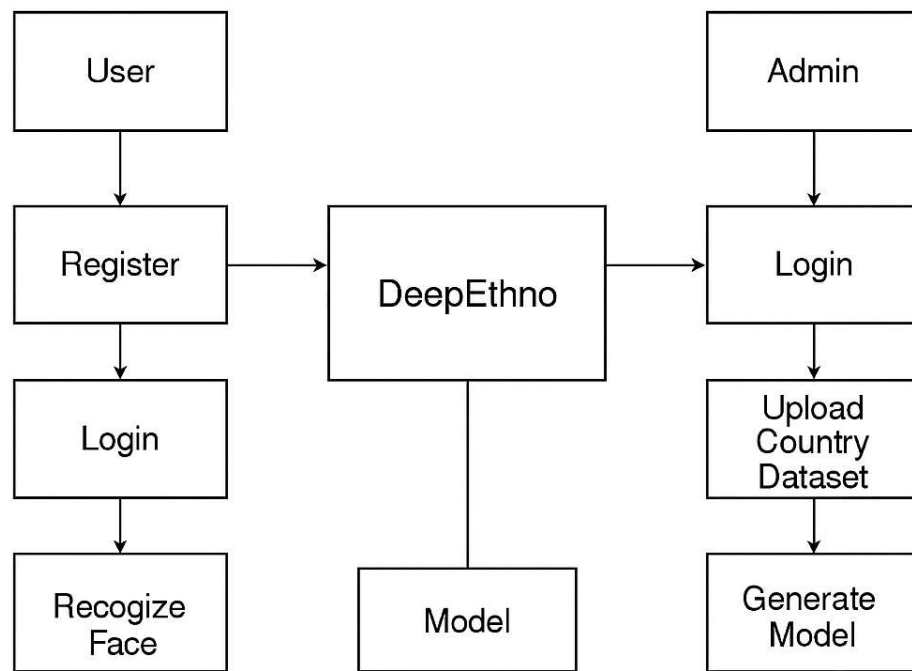


FIG 4.4.2 : DATA FLOW DIAGRAM LEVEL 1

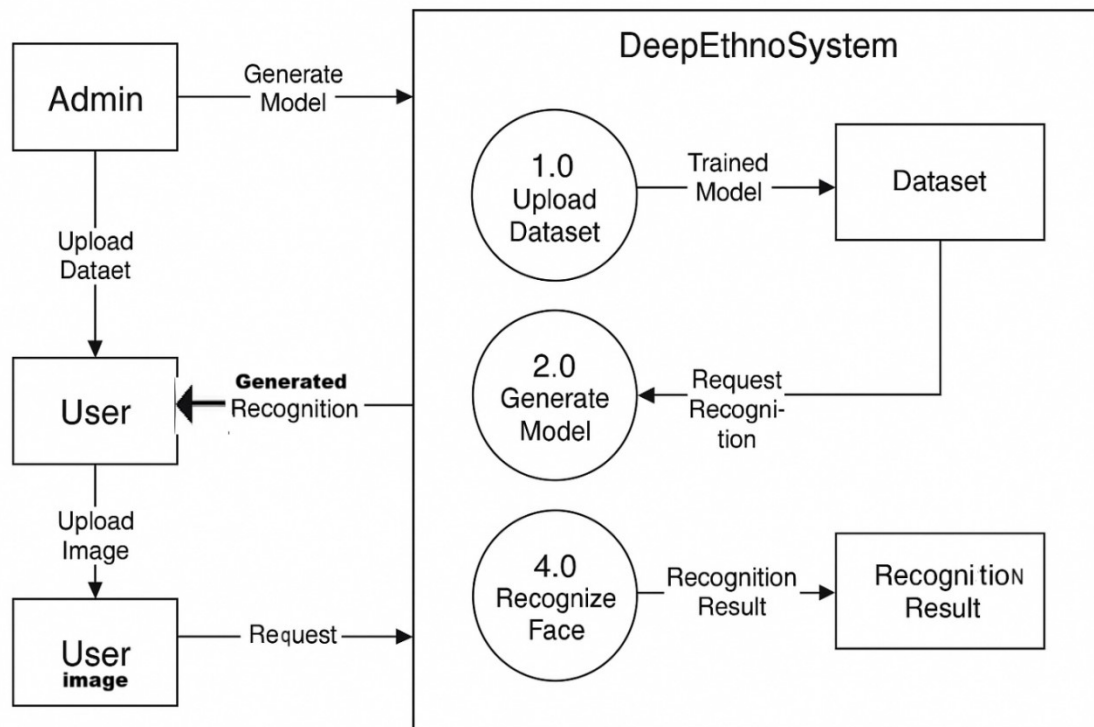


FIG 4.4.3 : DATA FLOW DIAGRAM LEVEL 2

4.5 UML DIAGRAMS

UML stands for Unified Modeling Language. UML is a standardized general-purpose modeling language in the field of object-oriented software engineering. The standard is managed, and was created by, the Object Management Group.

The goal is for UML to become a common language for creating models of object oriented computer software. In its current form UML is comprised of two major components: a Meta-model and a notation. In the future, some form of method or process may also be added to; or associated with, UML.

The UML represents a collection of best engineering practices that have proven successful in the modeling of large and complex systems.

The UML is a very important part of developing objects oriented software and the software development process. The UML uses mostly graphical notations to express the design of software projects.

GOALS:

The Primary goals in the design of the UML are as follows:

1. Provide users a ready-to-use, expressive visual modeling Language so that they can develop and exchange meaningful models.
2. Provide extendibility and specialization mechanisms to extend the core concepts.
3. Be independent of particular programming languages and development process.
4. Provide a formal basis for understanding the modeling language.
5. Encourage the growth of OO tools market.
6. Support higher level development concepts such as collaborations, frameworks, patterns and components.
7. Integrate best practices.

USE CASE DIAGRAM:

A use case diagram in the Unified Modeling Language (UML) is a type of behavioral diagram defined by and created from a Use-case analysis. Its purpose is to present a graphical overview of the functionality provided by a system in terms of actors, their goals (represented as use cases), and any dependencies between those use cases. The

main purpose of a use case diagram is to show what system functions are performed for which actor. Roles of the actors in the system can be depicted.

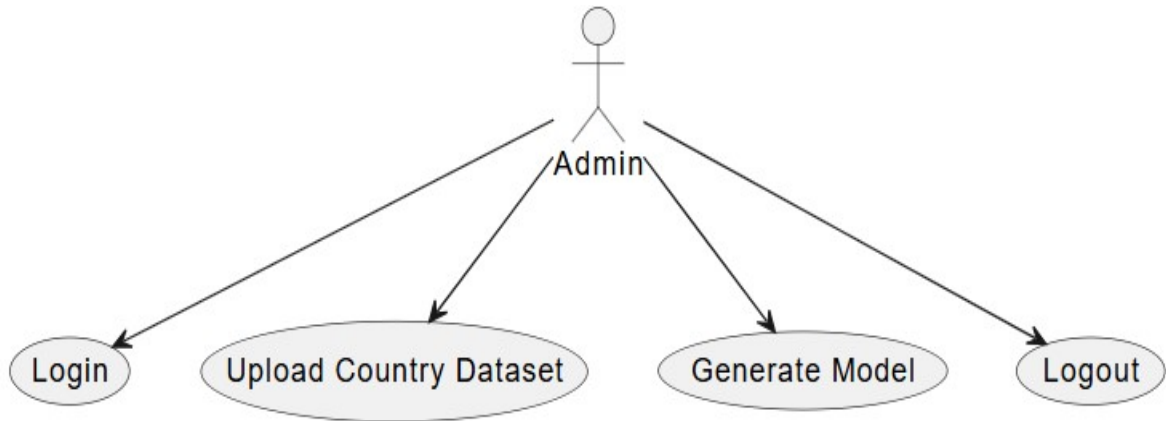


FIG 4.5.1 : ADMIN USE CASE DIAGRAM

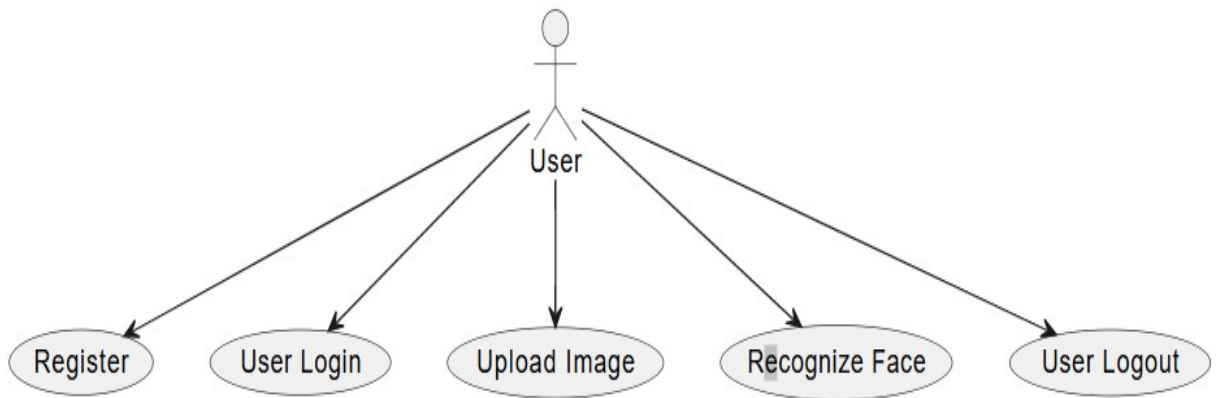


FIG 4.5.2: USER USE CASE DIAGRAM

CLASS DIAGRAM

In software engineering, a class diagram in the Unified Modeling Language (UML) is a type of static structure diagram that describes the structure of a system by showing the system's classes, their attributes, operations (or methods), and the relationships among the classes. It explains which class contains information.

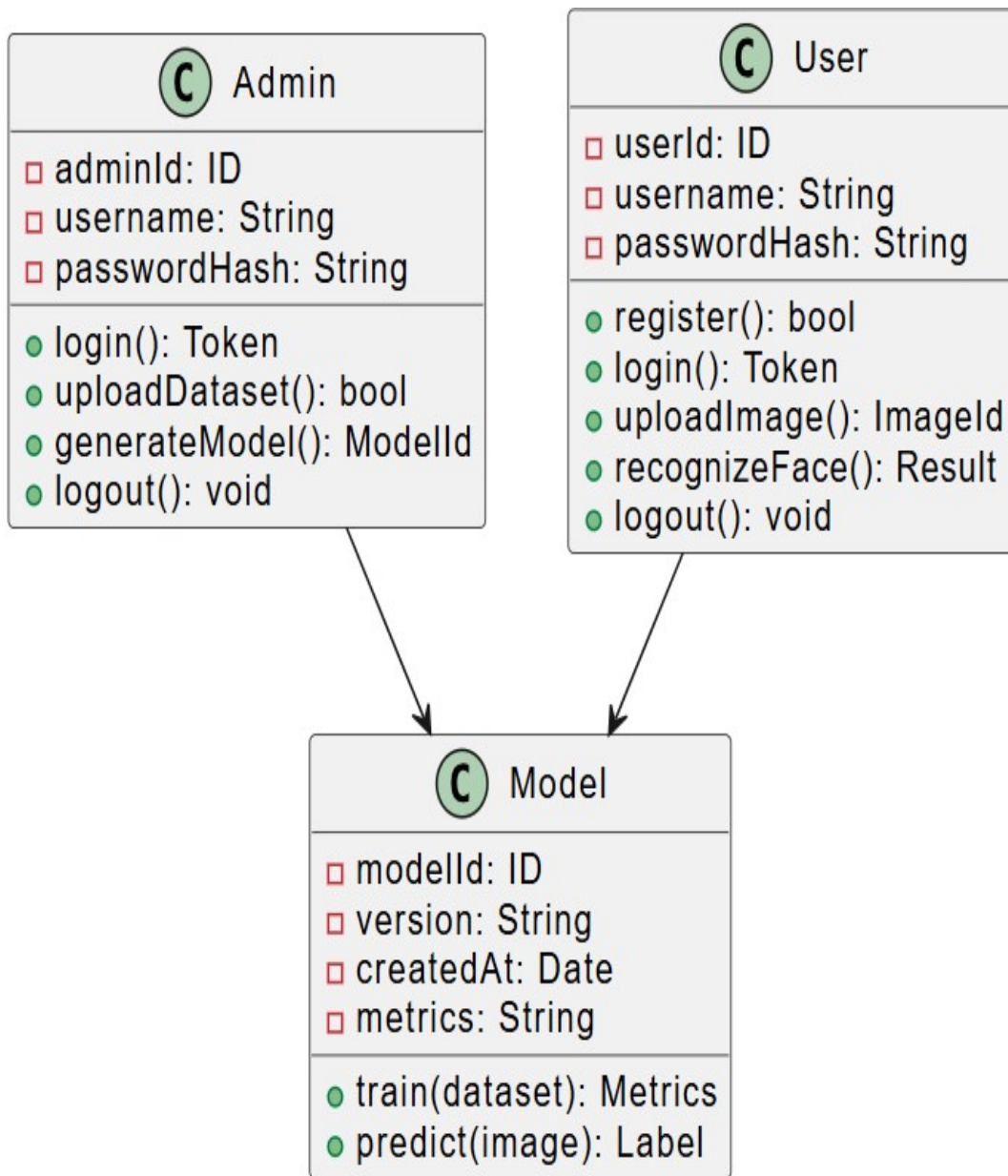


FIG 4.5.3: CLASS DIAGRAM

SEQUENCE DIAGRAM:

A sequence diagram in Unified Modeling Language (UML) is a kind of interaction diagram that shows how processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, event scenarios, and timing diagrams.

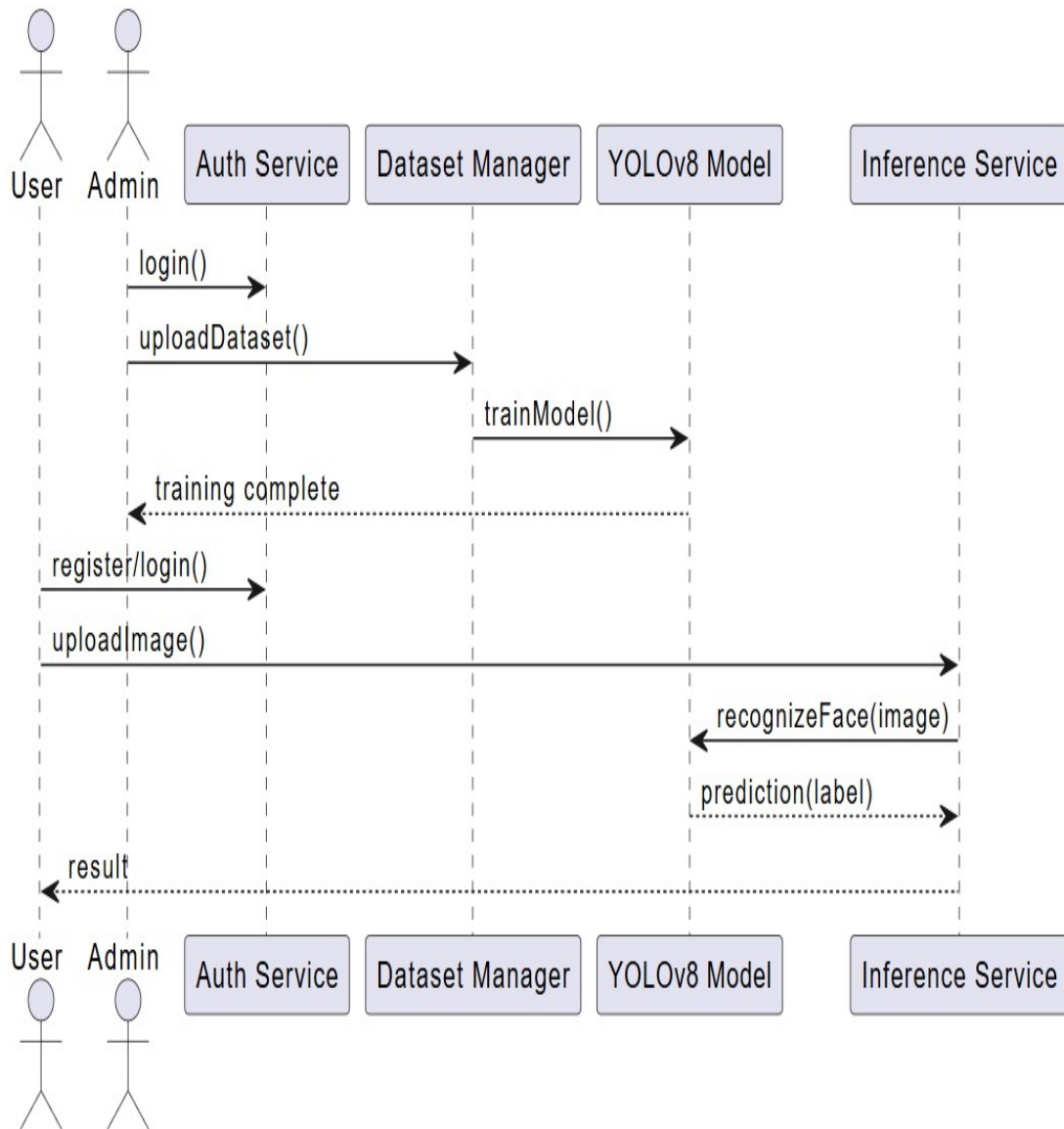


FIG 4.5.4 : SEQUENCE DIAGRAM

ACTIVITY DIAGRAM

An Activity Diagram is a type of UML diagram that represents the dynamic aspects of a system. It is primarily used to model the flow of control or activities in a process, system, or use case. This diagram is particularly useful in modeling the workflow of a system and how different actions or steps are performed, helping to visualize both sequential and parallel flows.

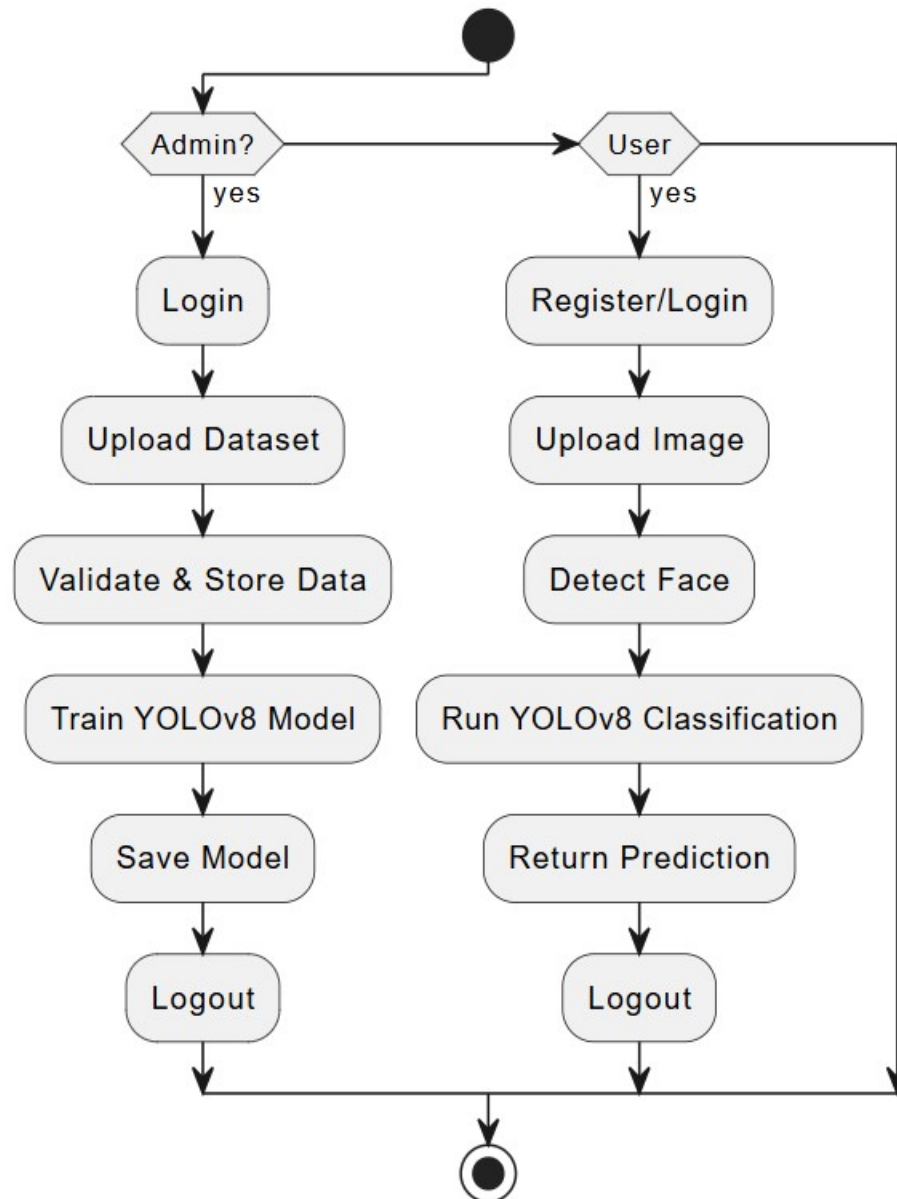


FIG 4.5.5: ACTIVITY DIAGRAM

DEPLOYMENT DIAGRAM

Deployment refers to the act of positioning something or someone in a strategic or ready state for a particular purpose, or the state of being positioned. In a military context, it means sending troops and equipment to a location for a mission, such as a war or a peaceful relief effort. In non-military contexts, such as business or software development, it can mean assigning people to a new location, spreading out a sales force, or making software available to users.

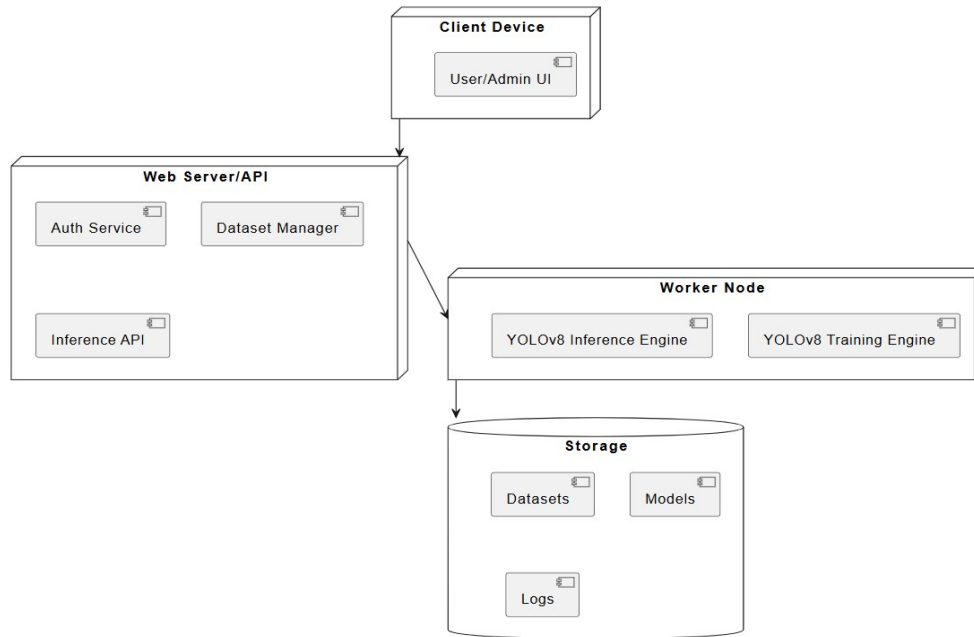


FIG 4.5.6: SYSTEM DEPLOYMENT DIAGRAM

STATECHART DIAGRAM

A statechart diagram visually models the behavior of a system or object by showing its different states (conditions or phases) and the events or conditions that trigger transitions between those states, effectively depicting the object's entire life cycle from creation to termination

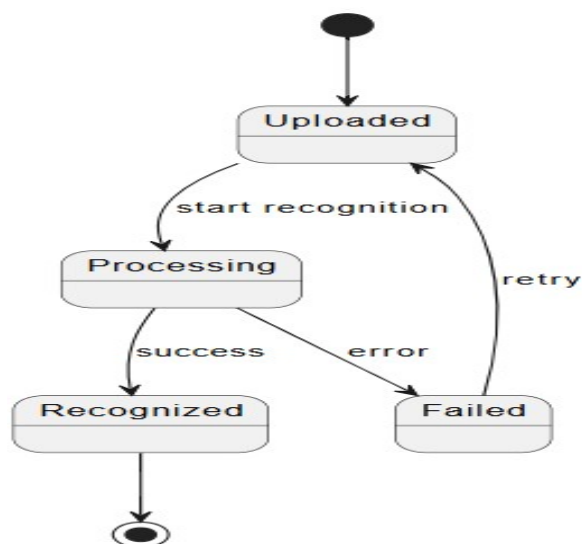


FIG 4.5.7: STATECHART DIAGRAM

COMPONENT DIAGRAM

A component is a fundamental part or element of a larger, more complex system, whether physical or abstract, that performs a specific function and has a defined interface. In technology, this could be a hardware device like a CPU, a software module, or even a firmware or virtual machine. In a broader sense, components are the constituent pieces that make up a whole, such as fresh fruit and vegetables being components of a healthy diet, or the specific parts and aspects that define something more complex.

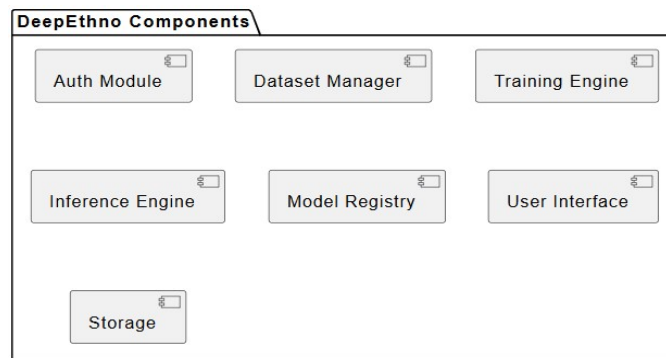


FIG 4.5.8 : SYSTEM COMPONENT DIAGRAM

OBJECT DIAGRAM

An object diagram represents a snapshot of objects at a particular moment, including their specific values and connections. It helps visualize how class relationships actually appear during program execution.

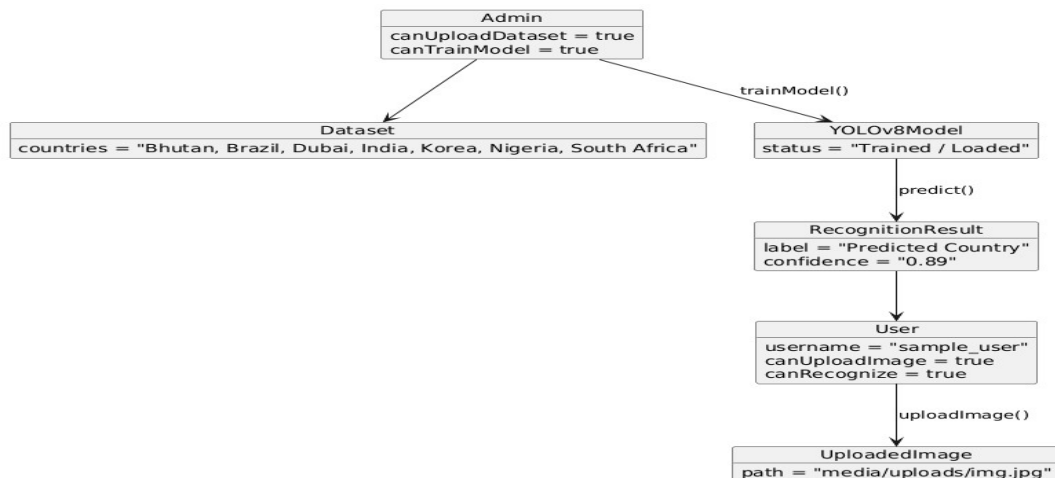


FIG 4.5.9 : OBJECT DIAGRAM

COLLABORATION DIAGRAM

A collaboration diagram focuses on the relationships and interactions between objects. Instead of time order, it highlights which objects communicate with each other and how messages flow within the network of objects.

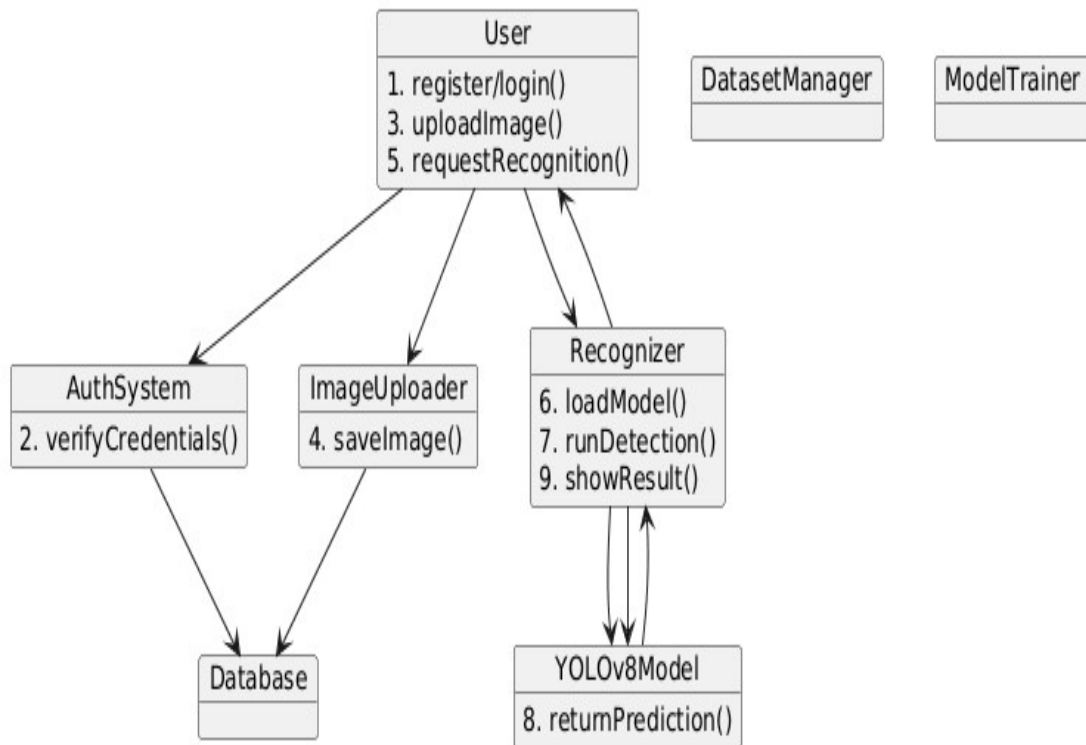


FIG 4.5.10 :COLLABORATION DIAGRAM (COMMUNICATION DIAGRAM)

4.4 INPUT AND OUTPUT DESIGN

➤ INPUT DESIGN

The input design is the link between the information system and the user. It comprises the developing specification and procedures for data preparation and those steps are necessary to put transaction data in to a usable form for processing can be achieved by inspecting the computer to read data from a written or printed document or it

can occur by having people keying the data directly into the system. The design of input focuses on controlling the amount of input required, controlling the errors, avoiding delay, avoiding extra steps and keeping the process simple. The input is designed in such a way so that it provides security and ease of use with retaining the privacy. Input Design considered the following things:

- What data should be given as input?
- How the data should be arranged or coded?
- The dialog to guide the operating personnel in providing input.
- Methods for preparing input validations and steps to follow when error occur.

➤ **OBJECTIVES**

Input Design is the process of converting a user-oriented description of the input into a computer-based system. This design is important to avoid errors in the data input process and show the correct direction to the management for getting correct information from the computerized system.

It is achieved by creating user-friendly screens for the data entry to handle large volume of data. The goal of designing input is to make data entry easier and to be free from errors. The data entry screen is designed in such a way that all the data manipulates can be performed. It also provides record viewing facilities.

When the data is entered it will check for its validity. Data can be entered with the help of screens. Appropriate messages are provided as when needed so that the user will not be in maize of instant. Thus the objective of input design is to create an input layout that is easy to follow

➤ **OUTPUT DESIGN**

A quality output is one, which meets the requirements of the end user and presents the information clearly. In any system results of processing are communicated to the users and to other system through outputs. In output design it is determined how the information is to be displaced for immediate need and also the hard copy output. It is the

most important and direct source information to the user. Efficient and intelligent output design improves the system's relationship to help user decision-making.

1. Designing computer output should proceed in an organized, well thought out manner; the right output must be developed while ensuring that each output element is designed so that people will find the system can use easily and effectively. When analysis design computer output, they should Identify the specific output that is needed to meet the requirements.

2. Select methods for presenting information.

3. Create document, report, or other formats that contain information produced by the system.

The output form of an information system should accomplish one or more of the following objectives.

- Convey information about past activities, current status or projections of the
- Future.
- Signal important events, opportunities, problems, or warnings.
- Trigger an action.
- Confirm an action.

4.6 PSUDO CODE

START

FUNCTION build_model():

 IF trained_model_exists("runs/train/deepethno/weights/best.pt"):

 LOAD YOLOv8 model with existing weights

 DISPLAY "Model Loaded Successfully"

 ELSE:

 INITIALIZE YOLOv8 base model (yolov8n.pt)

 TRAIN model using 'data.yaml' (7 country face images)

 SAVE best weights to 'runs/train/deepethno/weights/best.pt'

 DISPLAY "Model Trained Successfully"

FUNCTION upload_image():

 RECEIVE image from user

 SAVE image in 'media/uploads/'

 STORE image path and username in database

FUNCTION recognize():

 GET last uploaded image path from database

 LOAD trained YOLOv8 model

 RUN detection: results = model(image, conf=0.1)

 EXTRACT detected class labels (country names)

 FIND most common label → final_label

 DRAW bounding boxes on image

 SAVE result as 'Static/Result.jpg'

 DISPLAY final_label and output image on webpage

END

CHAPTER-5

IMPLEMENTATION

5.1 MODULE DESCRIPTION

➤ ADMIN

Here admin is a module should be login with the application, after successful login he can able

upload: we are uploading 7 countries faces images

{ 'Bhutan', 'Brazil', 'Dubai', 'India', 'Korean', 'Nigeriaa', 'South_Africa' }

Generate Model: based on countries dataset we are training yolo8 model

➤ USER

Here user is a module should register with the application, after the successful login he can **upload:** Upload trained person images.

Recognize: when I clicked on recognize it will load pretrained yolo8 algorithm and send the uploaded image to model then model will recognize and logout.

5.2 METHODOLOGY

Step 1: Dataset Collection

Facial image datasets from seven regions, Bhutan, Brazil, Dubai, India, Korea, Nigeria, and South Africa, are collected and organized into labeled folders for supervised training.

Step 2: Preprocessing

All images are resized, normalized, and labeled. A data.yaml file is prepared to map categories to YOLOv8. Dataset quality is checked to avoid imbalance or mislabeled images.

Step 3: YOLOv8 Model Training

The YOLOv8 base model is initialized. If previous training weights exist, they are loaded; otherwise, training is performed using the seven-region dataset. The model learns region-specific facial features and stores the best-performing weights.

Step 4: Model Saving and Deployment

The trained YOLOv8 model is saved in the runs/train directory. The model is then integrated into the web application for live inference.

Step 5: User Interaction Workflow

Users register and log in securely. They upload a facial image, which is stored in the server directory and passed to the YOLOv8 model for recognition.

Step 6: Recognition Process

YOLOv8 detects the face in the image, extracts features, classifies the most probable region, and outputs the prediction. The result image with bounding box and label is saved and displayed to the user.

Step 7: Result Logging

Prediction labels, image paths, and user details are stored in the database for tracking and academic evaluation.

➤ **PROPOSED ALGORITHM**

YOLOv8

YOLOv8 is used as the primary deep learning algorithm for both face detection and regional classification. It is trained on facial images from seven countries, learning discriminative facial features that enable accurate prediction of a person's regional origin. During inference, it detects the face in an uploaded image and classifies it into one of the predefined regional categories.

Image Preprocessing Algorithm

This algorithm standardizes all training and testing images by resizing, normalizing, and labeling them. It ensures that the YOLOv8 model receives clean and consistent input data, improving training stability, detection accuracy, and overall model performance.

Inference and Classification Algorithm

This algorithm loads the trained YOLOv8 model and processes user-uploaded images to perform real-time recognition. It extracts the detected class labels, determines the most probable regional category, draws bounding boxes, and outputs the final predicted region along with confidence scores.

5.3 SAMPLE CODE

Views.py

```
from django.shortcuts import render  
from django.core.files.storage import FileSystemStorage
```

```

import os
from django.conf import settings
import cv2
import pymysql
import traceback, sys, os
from ultralytics import YOLO
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt

def index(request):
    return render(request, 'index.html')

def admin_login(request):
    return render(request, 'admin/admin_login.html')

def login_action(request):
    username = request.POST.get('username')
    password = request.POST.get('password')
    if username == 'Admin' and password == 'Admin':
        return render(request, 'admin/admin_home.html')
    else:
        context = {'msg': 'Login failed..!'}
        return render(request, 'admin/admin_login.html', context)

def admin_home(request):
    return render(request, 'admin/admin_home.html')

def logout(request):
    return render(request, 'index.html')

def upload_dataset(request):
    return render(request, 'admin/upload_dataset.html')

def upload_action(request):
    global dataset
    dataset = "dataset\\"
    context = {'data': "Dataset Uploaded Successfully...!!"}

```

```

        return render(request, "admin/upload_dataset.html", context)
global train, valid
def generate_image(request):
    global train, valid
    train = "train"
    valid = "valid"
    context = {'train':len(train), 'validation':len(valid)}
    return render(request, 'admin/generate_image.html', context)
global model
def build_model(request):
    global train, valid, model
    if os.path.exists('runs/train/deepethno/weights/best.pt'):
        # Load the trained model
        model = YOLO("runs/train/deepethno/weights/best.pt") # Ensure path is
correct
        context = {"data": " Yolo Model Loaded ,Successfully...!!"}
        return render(request, 'admin/build_model.html', context)
    else:
        model = YOLO('yolov8n.pt')

        model.train(data='data.yaml', epochs=50,
                    imgsz=640,
                    batch=8,
                    name='deepethno',
                    project='runs/train'
                    )
        context = {"data": " Model Generated Successfully.."}
        return render(request, 'admin/build_model.html', context)
def user_registration(request):
    return render(request, 'user/user_registration.html')
def registration_action(request):

```

```

username = request.POST['username']
email = request.POST['email']
password = request.POST['password']
confirm_password = request.POST['confirm_password']
if password != confirm_password:
    return render(request, 'user/user_registration.html', {'msg': 'Passwords do
not match'})
con = pymysql.connect(
    host="localhost",
    user="root",
    password="root",
    database="deepethno",
    charset="utf8"
)
cur = con.cursor()
cur.execute("SELECT * FROM user WHERE username=%s OR email=%s",
(username, email))
existing_user = cur.fetchone()
if existing_user:
    con.close()
    return render(request, 'user/user_registration.html', {'msg': 'Username or
Email already exists!'})
cur.execute(
    "INSERT INTO user (username, email, password) VALUES (%s, %s, %s)",
    (username, email, password)
)
con.commit()
con.close()
return render(request, 'user/user_registration.html', {'msg': 'Registration
Successful!'})
def user_login(request):

```

```

        return render(request, 'user/user_login.html')
def user_login_action(request):
    username = request.POST['username']
    password = request.POST['password']
    con = pymysql.connect(host="localhost", user="root", password="root",
                           database="deepethno", charset='utf8')

    cur = con.cursor()
    cur.execute("SELECT * FROM user WHERE username=%s AND
password=%s", (username, password))
    user = cur.fetchone()
    con.close()

    if user:
        request.session['username']=username
        return render(request, 'user/user_home.html', {'username': username})
    else:
        return render(request, 'user/user_login.html', {'msg': 'Invalid username or
password'})
def user_home(request):
    return render(request, 'user/user_home.html')
def upload_image(request):
    return render(request, 'user/upload_image.html'

```

5.4 SYSTEM TESTING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

5.4.1 TYPES OF TESTS

➤ UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

➤ INTEGRATION TESTING

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields. Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

➤ FUNCTIONAL TEST

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

Valid Input	: identified classes of valid input must be accepted.
Invalid Input	: identified classes of invalid input must be rejected.
Functions	: identified functions must be exercised.
Output	:identified classes of application outputs must be exercised.

Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

➤ **SYSTEM TEST**

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

➤ **WHITE BOX TESTING**

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

➤ **BLACK BOX TESTING**

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document. It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

➤ **UNIT TESTING**

Unit testing is usually conducted as part of a combined code and unit test phase of the software lifecycle, although it is not uncommon for coding and unit testing to be conducted as two distinct phases.

5.4.2 TEST STRATEGY AND APPROACH

Field testing will be performed manually and functional tests will be written in detail.

TEST OBJECTIVES

- All field entries must work properly.
- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

FEATURES TO BE TESTED

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

➤ INTEGRATION TESTING

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects.

The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

➤ ACCEPTANCE TESTING

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

CHAPTER-6

RESULT AND DISCUSSION

6.1 OUTPUT SCREENSHOTS

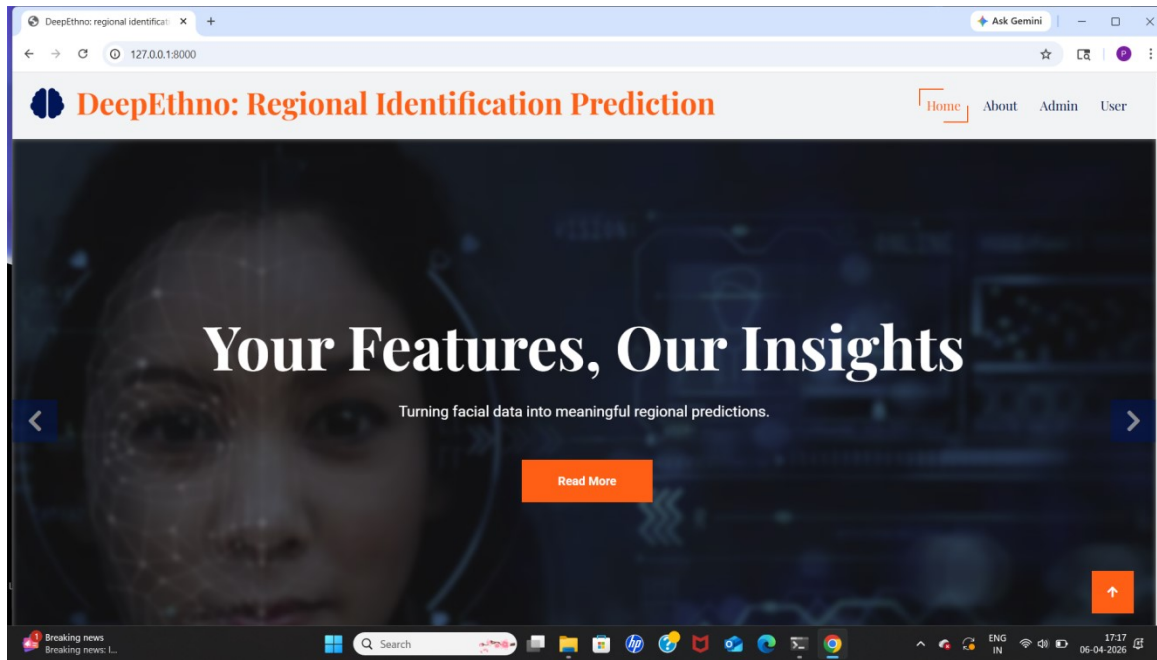


FIG 6.5.1 : HOME INDEX PAGE

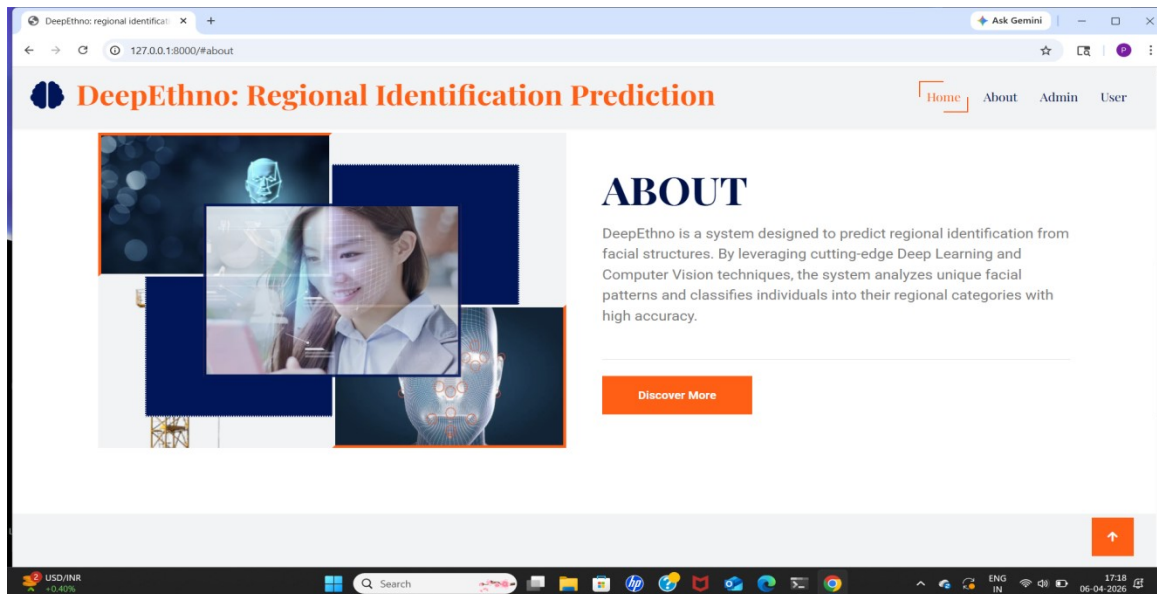


FIG 6.5.2 : ABOUT US PAGE

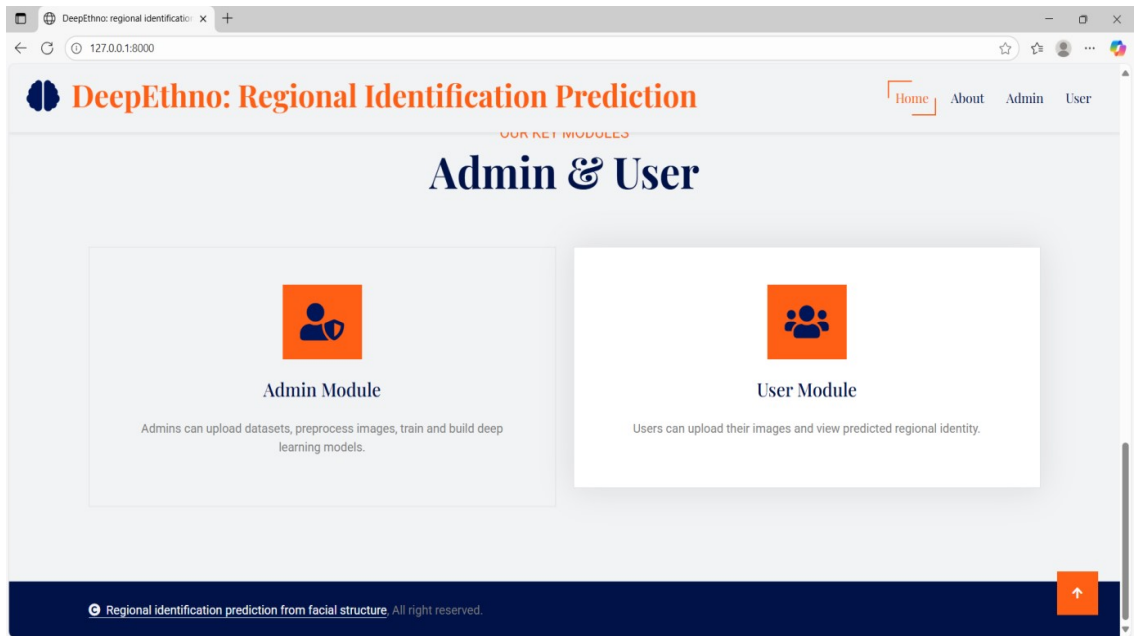


FIG 6.5.3 : ADMIN AND USER LOGIN PAGE

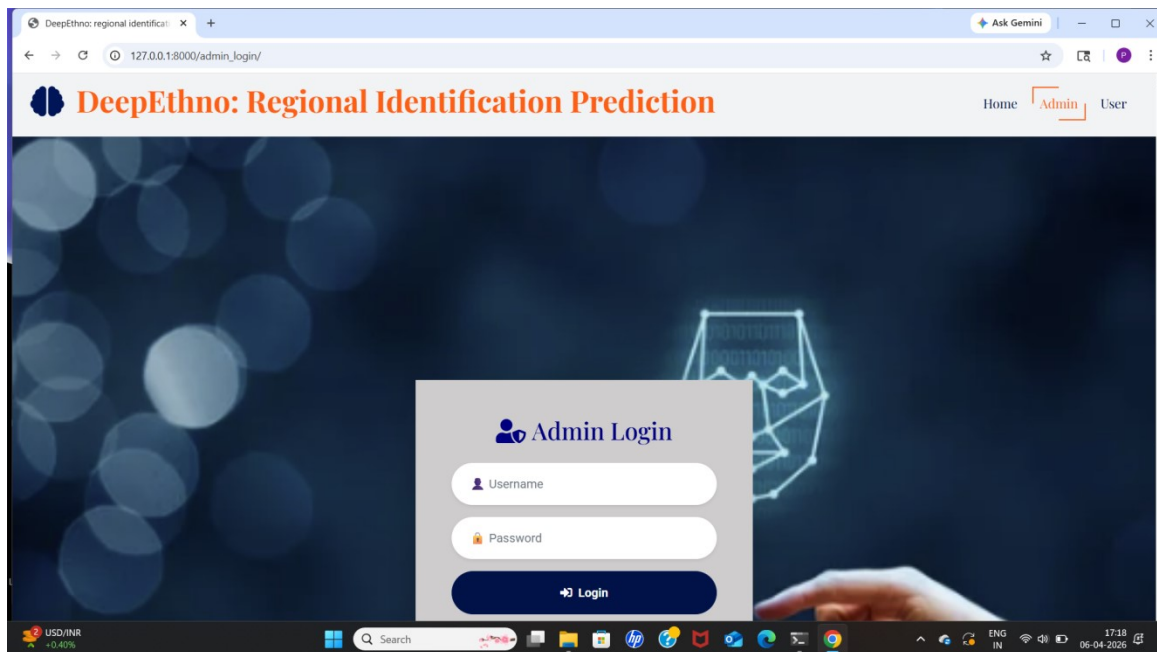


FIG 6.5.4 : ADMIN LOGIN

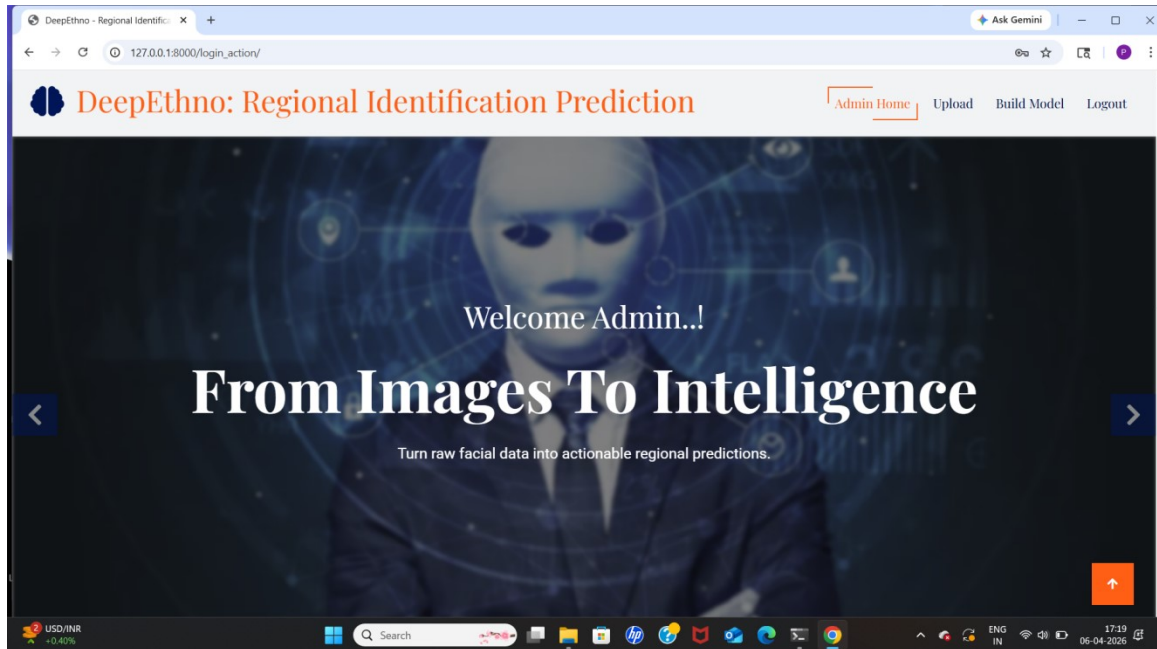


FIG 6.5.5 : ADMIN HOME

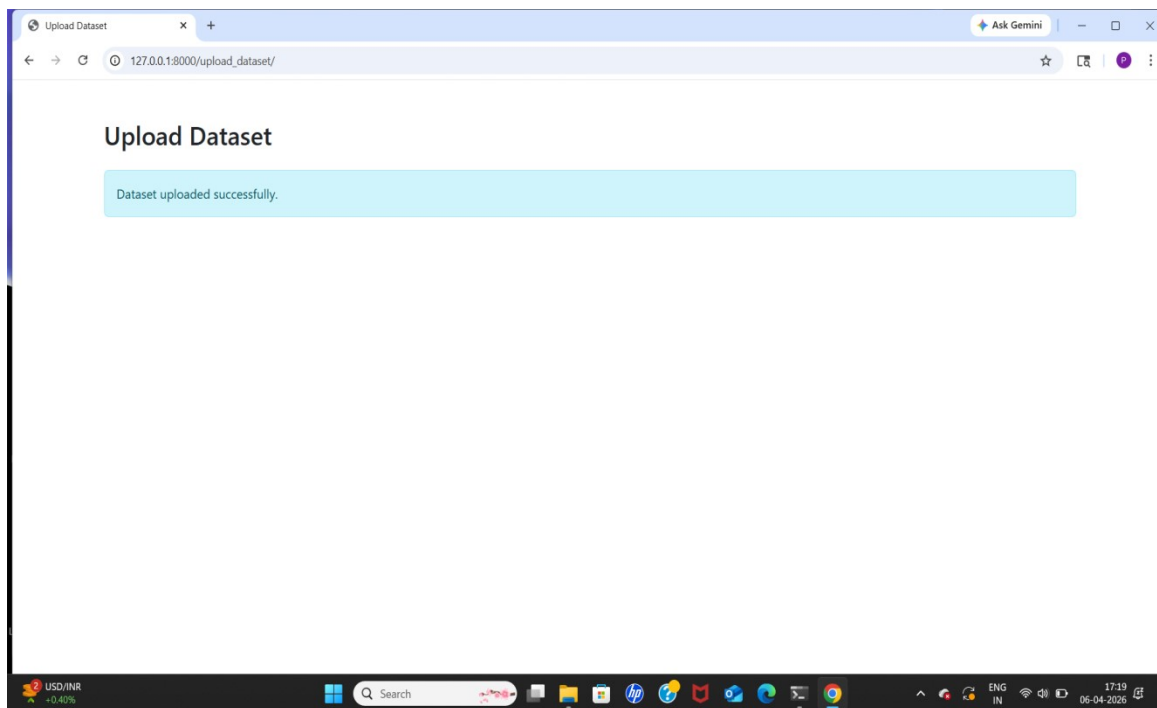


FIG 6.5.6 : UPLOADING DATASET

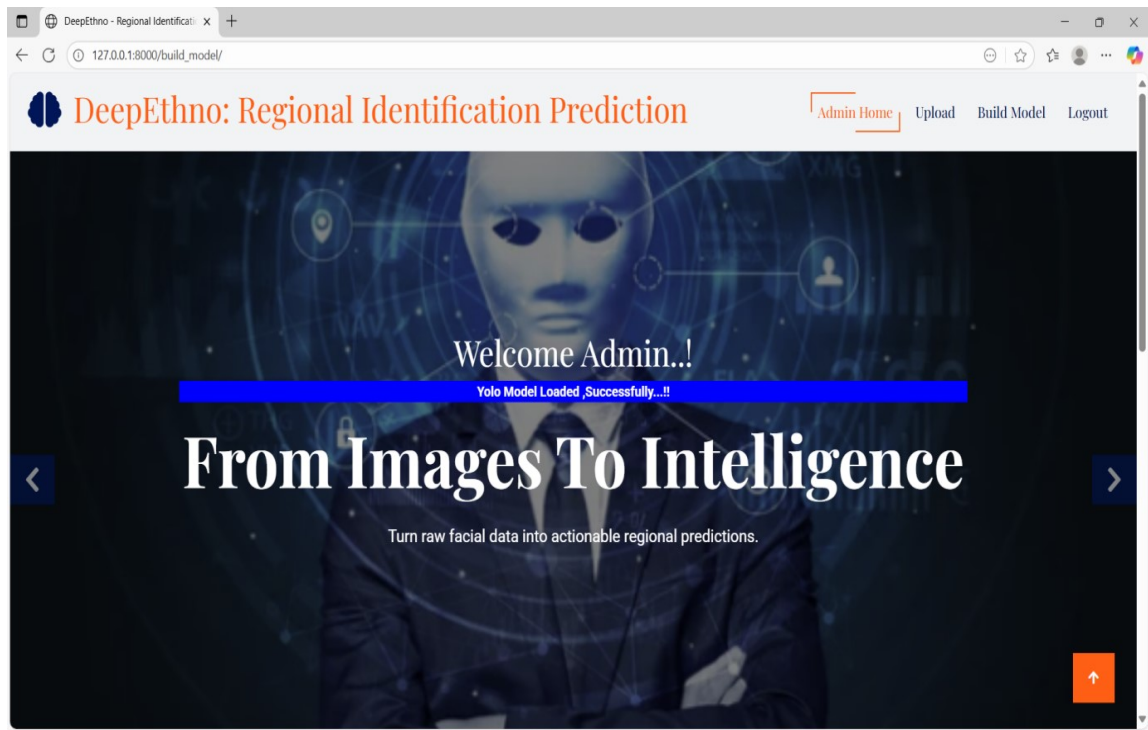


FIG 6.5.7 : MODEL LOADED

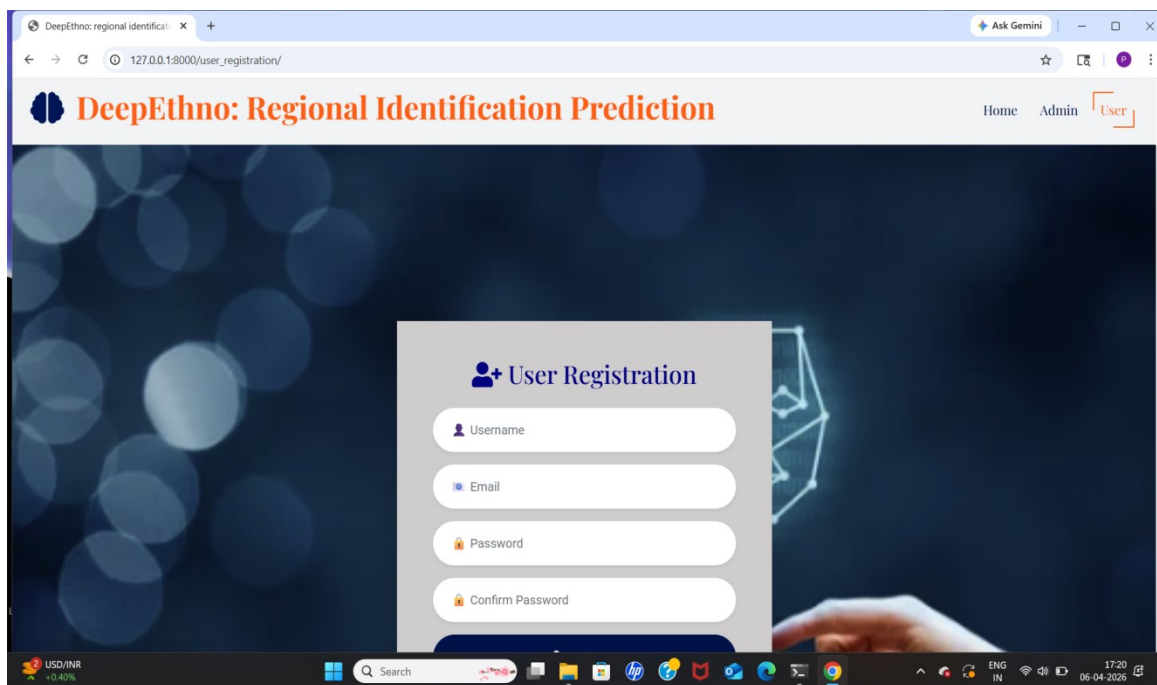


FIG 6.5.8 : REGISTRATION PAGE

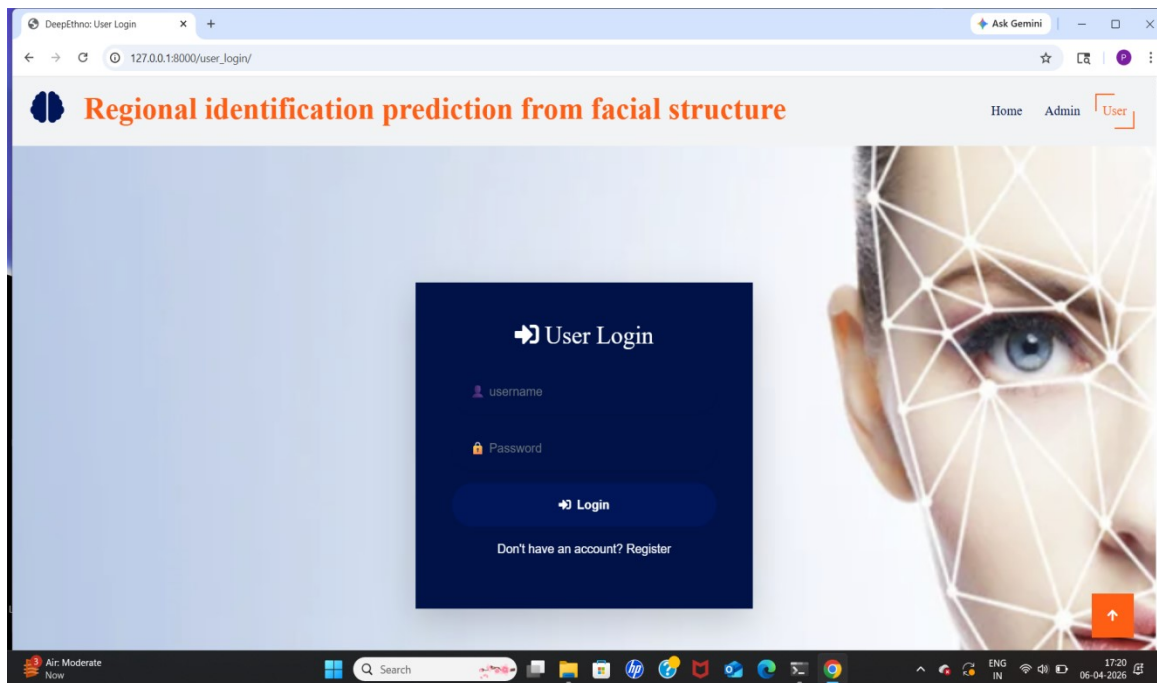


FIG 6.5.9 : LOGIN PAGE

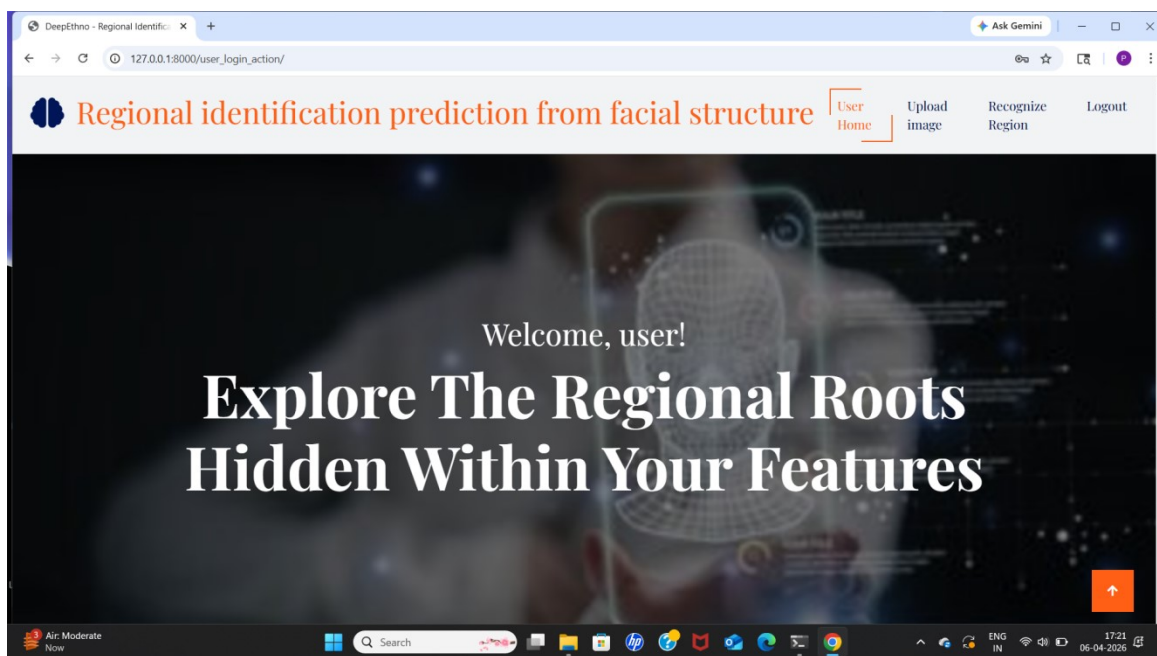


FIG 6.5.10 : USER HOME

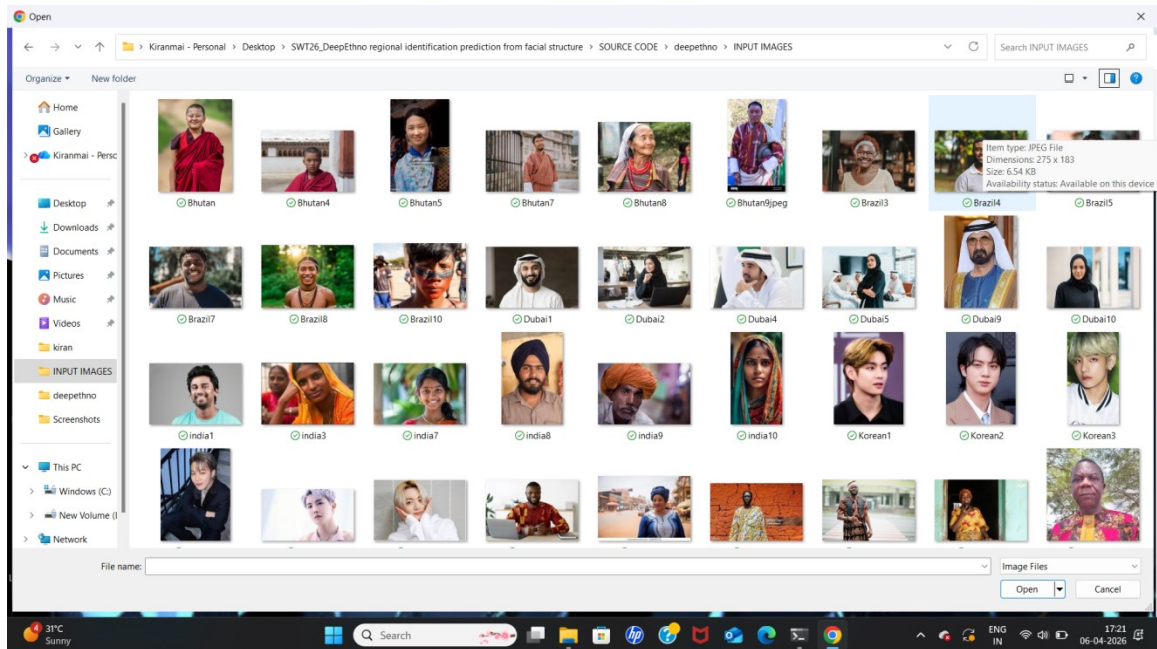


FIG 6.5.11 : UPLOADING IMAGE

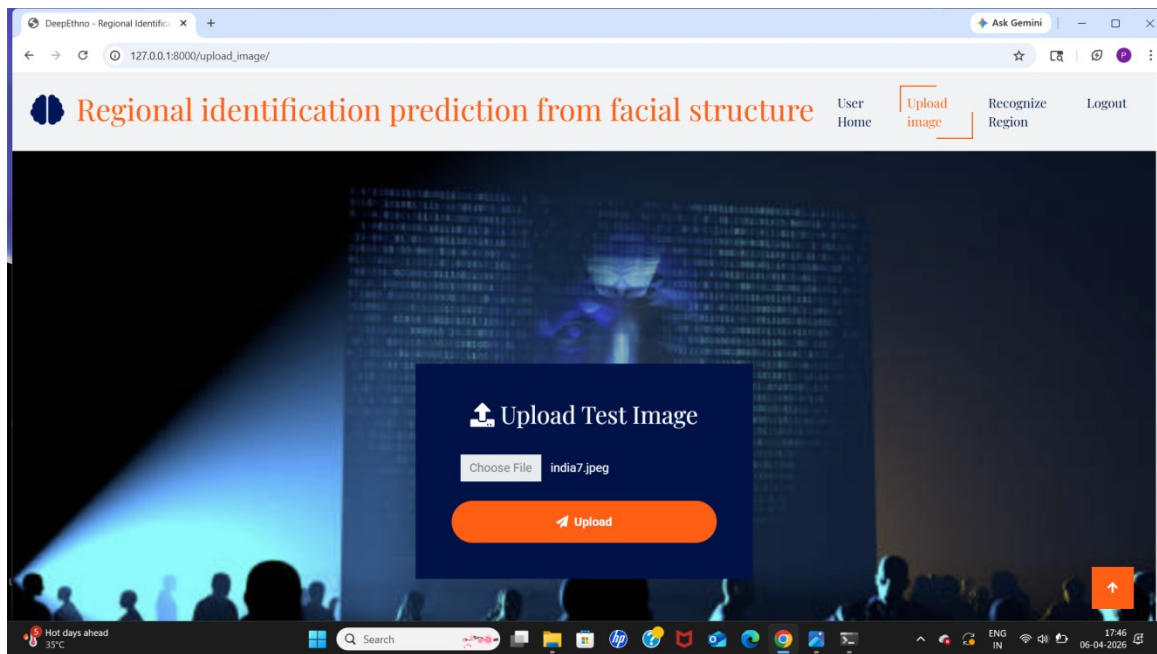


FIG 6.5.12 : FINGER PRINT UPLOADED

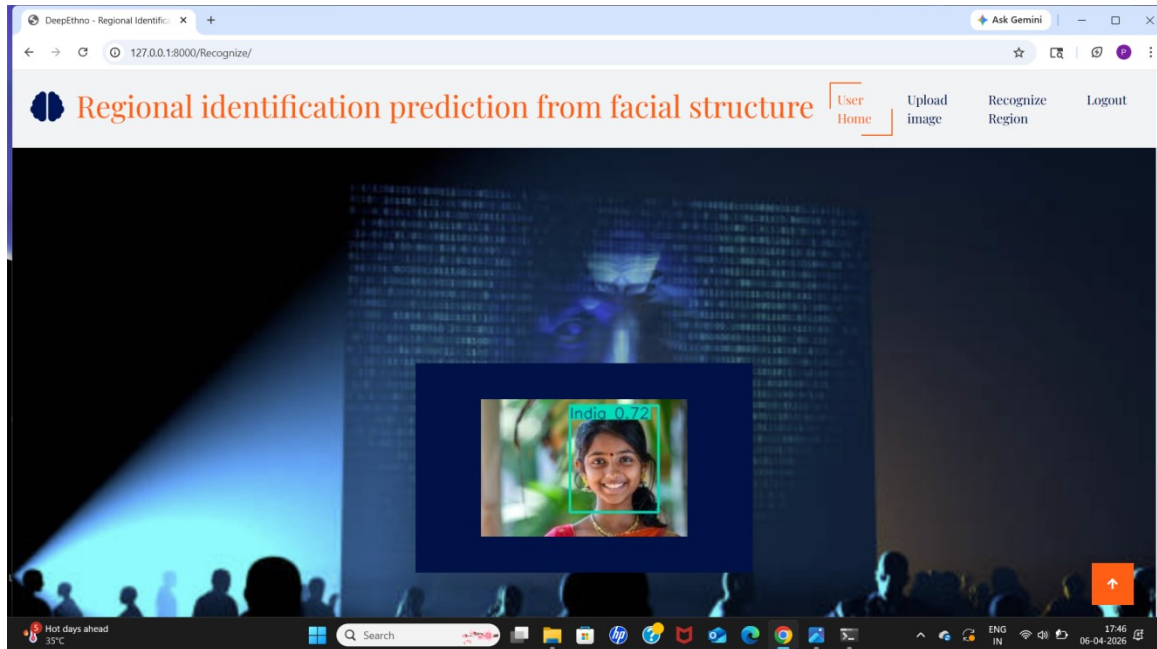


FIG 6.5.13 : RESULT 1

6.2 RESULT

The YOLOv8-based regional identification system successfully classifies facial images into seven regional categories with high accuracy and real-time performance. The admin module effectively manages dataset upload and model generation, ensuring reproducible training workflows. Users are able to register, log in, upload images, and receive regional predictions with bounding boxes and confidence scores. The system demonstrates strong detection capability even with diverse facial structures and varying lighting conditions. The modular design supports dataset expansion, model updates, and controlled usage. Overall, the system provides an efficient, secure, and academically valuable framework for regional facial recognition research.

CHAPTER-7

CONCLUSION AND FUTURE SCOPE

7.1 CONCLUSION

The DeepEthno system demonstrates the potential of applying modern deep-learning architectures, specifically YOLOv8, to the task of regional identification based on facial structures. By leveraging datasets from diverse populations such as Bhutan, Brazil, Dubai, India, Korea, Nigeria, and South Africa, the system is able to learn distinctive structural and morphological cues that differentiate individuals across regions. The modular framework integrates dataset management, model training, and user-facing recognition into a cohesive academic prototype, thereby providing both technical robustness and practical usability.

The research highlights not only the technical feasibility but also the importance of fairness, reproducibility, and ethical considerations in demographic recognition systems. Compared to traditional recognition systems, DeepEthno benefits from YOLOv8's anchor-free detection and classification capabilities, achieving both high accuracy and real-time inference speed. At the same time, the study acknowledges the limitations arising from dataset imbalance, cross-cultural biases, and the potential misuse of demographic recognition technologies.

Ultimately, DeepEthno serves as an academic contribution rather than a deployable commercial product. Its value lies in demonstrating technical innovation while raising awareness about the ethical and fairness-related challenges of demographic prediction systems. The project establishes a foundation for future research in fairness-aware computer vision and reproducible deep-learning workflows, ensuring that academic prototypes are responsibly documented and clearly distinguished from real-world deployments.

7.2 FUTURE SCOPE

The future direction of DeepEthno research involves both **technical enhancements** and **ethical safeguards**. On the technical side, the system can be

expanded with larger and more balanced datasets covering additional countries and ethnicities to reduce bias and improve generalization. Advanced augmentation techniques, such as generative adversarial networks (GANs) for synthetic face generation, can help fill underrepresented groups and improve fairness in training. Incorporating explainable AI (XAI) methods will enable greater interpretability by showing which facial features drive classification decisions, helping to validate results and expose potential biases.

In terms of deployment, the model can be optimized for cross-platform compatibility using lightweight formats such as TensorRT, CoreML, or ONNX, making it accessible for academic demonstrations across mobile and edge devices. Another promising direction is to explore **multimodal systems** by combining facial cues with contextual information (such as speech, clothing, or cultural artifacts) to enhance classification robustness. Privacy-preserving machine learning approaches like federated learning and encrypted inference should also be investigated to ensure that sensitive biometric data is never exposed.

From an ethical perspective, the project must include comprehensive fairness evaluations, per-region performance reporting, and strong disclaimers about academic-only use. Collaborative efforts with sociologists, ethicists, and policymakers will also enrich the system by integrating cultural awareness and mitigating potential misuse. Long term, the DeepEthno framework can evolve into a broader research platform for fairness-aware biometric analysis, contributing to global discussions about responsible AI in demographic recognition.

REFERENCES

1. Ultralytics, “YOLOv8 — Models & Usage,” Ultralytics Docs, Jan. 2023.
2. D. Reis, “Real-Time Flying Object Detection with YOLOv8,” *arXiv preprint*, May 2023.
3. N. Ladi, “Running Ultralytics object detection and segmentation models in a few lines of code,” Ultralytics Blog, Jun. 2024.
4. Z. Zhang et al., “Validating Privacy-Preserving Face Recognition under a Minimum Assumption,” *Proc. CVPR*, 2024.
5. A. Sumsion, “Surveying Racial Bias in Facial Recognition: Balancing Accuracy and Fairness,” *Electronics*, 2024.
6. Q. Zhang et al., “Understanding of facial features in face perception: insights for computational models,” *Frontiers in Computational Neuroscience*, 2024.
7. “What is YOLOv8: An In-Depth Exploration,” *arXiv preprint*, Aug. 2024.
8. Roboflow, “What is YOLOv8? A Complete Guide,” Roboflow Blog, Oct. 2024.
9. S. Mupparaju et al., “A Review on YOLOv8 and Its Advancements,” ResearchGate, 2024.
10. “Review of Demographic Fairness in Face Recognition,” *arXiv preprint*, 2025.